

KONTRAK KERJA DAN SPESIFIKASI ANTARMUKA

Tugas Kelompok 1 Analisis Numerik Gasal 2026/2027 — Soal Nomor 2 (SETAR)

Metode Khusus Kelompok Genap: Givens Rotations & Persamaan Normal

1. Prinsip Kolaborasi Paralel

Untuk memaksimalkan efisiensi dan memastikan pengerjaan dapat berjalan **100% paralel tanpa saling menunggu**, tugas Nomor 2 dibagi menjadi dua peran teknis yang independen namun komplementer. Person A (Solver & Teori) langsung menguji kodenya menggunakan **matriks toy di Halaman 10 PDF**, sementara Person B (Data & Finansial) memproses dataset `stock_train.csv` dan `stock_test.csv`. Keduanya terikat oleh antarmuka fungsi (interface) di bawah ini.

Aspek	Person A (Kamu) Numerical & Algorithm Engineer	Person B (Temanmu) Data & Financial Modeling
Sub-Poin Tugas	Poin iii, iv, v + Pseudocode	Poin i, ii, vi, vii + README.md
Fokus Coding	- Solver Persamaan Normal dari nol - Solver QR Givens Rotations dari nol - Back-substitution dari nol - Verifikasi eliminasi awal (matriks G_k)	- Preprocessing CSV & hitung return - Konstruksi matriks desain A dan b - Evaluasi out-of-sample ke test set - Perhitungan metrik RMSE
Fokus Analisis & Laporan	- Analisis <i>condition number</i> $\kappa_2(A^T A)$ - Perbandingan FLOPs & konsumsi memori - Teori dampak outlier ekstrem (L_2 norm) - Pseudocode kedua algoritma	- Formulasi matematis & dimensi sistem - Investigasi isu skala/kondisi matriks A - Visualisasi grafik overlay time series - Interpretasi finansial koefisien (momentum vs mean-reverting)
Berkas Utama	<code>src/solvers.py</code> <code>test_toy.py</code>	<code>src/data_pipeline.py</code> <code>README.md</code>

2. Kontrak 1: Person B → Person A (Data Pipeline)

Person B mengimplementasikan modul `src/data_pipeline.py`. Matriks A harus memiliki **6 kolom dengan urutan baku** sesuai Persamaan (5) pada dokumen tugas, agar koefisien solusi x selaras dengan parameter model:

```
# File: src/data_pipeline.py
import numpy as np

def build_dataset(csv_path: str) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Membaca CSV saham, menghitung return, dan menyusun matriks overdetermined A &
    target b.

    Output:
    - A: np.ndarray (shape=(M, 6), dtype=float64)
      Kolom 0: I_t           (1 jika R_{t-1} >= 0, else 0)
      Kolom 1: I_t * R_{t-1} (Lag-1 Bullish)
      Kolom 2: I_t * R_{t-2} (Lag-2 Bullish)
      Kolom 3: 1 - I_t       (1 jika R_{t-1} < 0, else 0)
      Kolom 4: (1 - I_t)*R_{t-1} (Lag-1 Bearish)
      Kolom 5: (1 - I_t)*R_{t-2} (Lag-2 Bearish)
    - b: np.ndarray (shape=(M, ), dtype=float64) -> Target return hari ke-t (R_t)
```

- actual_returns: np.ndarray (shape=(Total,), dtype=float64) -> deret return utuh untuk plot

Ketentuan:

- Return $R_t = (P_t - P_{t-1}) / P_{t-1}$
 - Karena lag p=2, baris dimulai dari observasi t=3 (indeks 2 pada 0-indexed).
 - Jika ada N data harga, jumlah baris $M = N - 3$.
- """

3. Kontrak 2: Person A → Person B (Numerical Solvers)

Person A mengimplementasikan modul src/solvers.py tanpa menggunakan pustaka penyelesaian SPL/LSP bawaan. Output berupa dictionary terstruktur yang langsung dapat dikonsumsi oleh script evaluasi Person B:

```
# File: src/solvers.py
import numpy as np

def solve_normal_equations(A: np.ndarray, b: np.ndarray) -> dict:
    """
    Menyelesaikan Least Squares Problem via Persamaan Normal:  $A^T A x = A^T b$ .
    Returns:
    {
        'x': np.ndarray (shape=(6,)),          # Solusi koefisien
        'residual_norm': float,                 #  $\|A^T x - b\|_2$ 
        'cond_A': float,                       #  $\kappa_2(A)$ 
        'cond_ATA': float                      #  $\kappa_2(A^T A)$ 
    }
    """

def solve_givens_qr(A: np.ndarray, b: np.ndarray) -> dict:
    """
    Menyelesaikan Least Squares Problem via Givens Rotations QR + Back Substitution.
    Returns:
    {
        'x': np.ndarray (shape=(6,)),          # Solusi kuadrat terkecil x_LS
        'residual_norm': float,                 #  $\|A^T x_{LS} - b\|_2$ 
        'R': np.ndarray (shape=(6, 6)),        # Matriks segitiga atas R
        'G1': np.ndarray                       # Matriks rotasi pertama
    }
    pengeliminasi subdiagonal
    """

def predict(A: np.ndarray, x: np.ndarray) -> np.ndarray:
    """Menghitung  $R_{hat} = A @ x$  untuk data train/test."""
    return A @ x

def calculate_rmse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Menghitung RMSE =  $\sqrt{\text{mean}((y_{true} - y_{pred})^2)}$ ."""
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))
```

4. Integrasi Akhir (Digunakan Bersama di main.py)

```
# File: main.py
from src.data_pipeline import build_dataset
from src.solvers import solve_normal_equations, solve_givens_qr, predict,
calculate_rmse

# 1. Pipeline Data (Hasil Person B)
A_train, b_train, ret_train = build_dataset("data/stock_train.csv")
A_test, b_test, ret_test    = build_dataset("data/stock_test.csv")

# 2. Eksekusi Solver (Hasil Person A)
res_normal = solve_normal_equations(A_train, b_train)
res_givens = solve_givens_qr(A_train, b_train)

# 3. Evaluasi Out-of-Sample (Person B)
x_opt = res_givens['x']
rmse_train = calculate_rmse(b_train, predict(A_train, x_opt))
rmse_test  = calculate_rmse(b_test, predict(A_test, x_opt))
print(f"Evaluasi Model: RMSE Train = {rmse_train:.6f} | RMSE Test = {rmse_test:.6f}")
```

5. Roadmap Eksekusi & Check-List Laporan

- **Fase 1 (Mandiri):** Person A buat & tes solver di test_toy.py pakai contoh Hal. 10. Person B buat data_pipeline.py dan cek distribusi return.
- **Fase 2 (Integrasi):** Gabungkan di main.py, hitung RMSE, catat waktu & memori, generate plot overlay kontinu.
- **Fase 3 (Laporan):** Pastikan memuat *Pseudocode* (Person A), *Tabel Performa & Kondisi Matriks* (Person A), *Grafik Overlay Train-Test* (Person B), *Interpretasi Finansial* (Person B), dan *README.md*.